Plugins

Eleanor exposes four extension points that share a single plugin model: a generic in-process registry, lazy entry-point discovery, and a common collision/override policy. The registry primitive lives at `eleanor.plugin.PluginRegistry`; each extension point instantiates it with its own spec shape and built-in name set.

Extension points

Kind	Entry-point group	Spec type
Executor	`eleanor.executors`	`ConfigurablePluginSpec`
Kernel	`eleanor.kernels`	`ConfigurablePluginSpec`
Navigator	`eleanor.navigators`	`SimplePluginSpec`
Output sink	`eleanor.outputs`	`ConfigurablePluginSpec`

Every entry point must resolve to a `SimplePluginSpec` or `ConfigurablePluginSpec` instance (from `eleanor.plugin`). Bare callables are not accepted.

Names are short strings (e.g. `random`, `eq36`, `dask`). Built-in names are reserved and cannot be overridden through the registry; see Collision policy below.

Configuration shape

Kernel, navigator, and output sink selection all use the same flat `kind`-keyed shape. The `kind` key selects the plugin; all remaining keys are forwarded to the plugin's `parse_settings` as a flat dict:

```yaml
kernel:
  kind: eq36
  model: b-dot
  charge_balance: H+

navigator:
  kind: random

output:
  kind: postgres
  database:
    host: localhost
    port: 5432
    database: eleanor
```

The short-string form `navigator: random` is accepted as sugar for `{kind: random}`.

`output` additionally accepts a **list** of such blocks, one per sink, each with an optional `name` (defaulting to `kind`). `name` is Eleanor's, not the plugin's: it is stripped alongside `kind` before the rest is forwarded to `parse_settings`.

```yaml
output:
  - kind: postgres
    database: {host: localhost, database: eleanor}
  - kind: csv
    name: export
    filename: summary.csv
    query: {...}
```

Navigator contract

Navigator plugins must expose:

- `num_systems(order: Order, scale: int) -> int`: total number of VS points the navigator will produce for the given scale.
- `navigate(order: Order, kernel: AbstractKernel, scale: int, batch_size: int, *args, **kwargs) -> Iterator[list[vs.Point]]`: a batch iterator that yields lists of up to `batch_size` points.

Eleanor passes `max_attempts` as a keyword argument to `navigate`; plugins should accept `**kwargs` to tolerate future additions. Points carry no order id: the id belongs to the output sink, which receives it as a parameter to `prepare_batch` / `commit_batch`, so a navigator never needs to know it.

Across a complete `navigate(...)` iteration, the total number of yielded points must match `num_systems(order, scale)`. Eleanor raises an error if the counts diverge.

Distributing a third-party plugin

Third-party plugins register themselves through Python entry points declared in the distribution's `pyproject.toml`:

```toml
[project.entry-points."eleanor.navigators"]
my_nav = "my_project.navigators:my_nav_spec"

[project.entry-points."eleanor.kernels"]
my_kernel = "my_project.kernels:my_kernel_spec"

[project.entry-points."eleanor.outputs"]
my_sink = "my_project.outputs:my_sink_spec"
```

Discovery runs lazily on the first call to `available_*()` or `get_*()` for the relevant extension point. Any entry-point load or validation failure is a hard error — a broken plugin fails fast rather than silently disappearing from the available set.

End-to-end example: third-party output sink package

Minimal package layout:

```
my_sink_plugin/
  pyproject.toml
  src/my_sink_plugin/output.py
```

`pyproject.toml`:

```toml
[project]
name = "my-sink-plugin"
version = "0.1.0"
dependencies = ["eleanor>=0.0.0"]

[project.entry-points."eleanor.outputs"]
my_sink = "my_sink_plugin.output:my_sink_spec"
```

`src/my_sink_plugin/output.py`:

`AbstractOutputSink` is generic in the type of the order id it hands out: `AbstractOutputSink[IdT]`. A sink chooses its own id space and pins it when it subclasses – `AbstractOutputSink[int]` for a database sequence, `AbstractOutputSink[UUID]` for a sink with nothing to draw a sequence from. `begin_run` returns a value of that type and Eleanor hands the same value back to `prepare_batch` and `commit_batch`; it never constructs or inspects one, so nothing constrains the type beyond being picklable.

Persisting a batch is split into two halves. `prepare_batch` always runs in a worker process, with the whole compute graph available, and reduces it to whatever compact form the sink wants. `commit_batch` durably persists that payload – in the worker if `supports_worker_commit()` returns `True`, in the parent otherwise.

The point of the split is that only the prepared payload crosses the process boundary, never the ~100k-object compute graph. What you return from `prepare_batch` is private to your sink; Eleanor never inspects it, and types it as `object`. The two requirements are that it pickles, and that it is *cheaper to unpickle* than the graph. That last part is about object count rather than byte count, so a columnar payload (one array per column) does far better than a row-oriented one, which keeps the graph's object count intact.

Because the sink itself is pickled into a worker once per chunk, your sink class must be importable by name, and `prepare_batch` must not depend on mutating sink state – the worker's copy is discarded.

Not under every executor, though: `serial` runs `prepare_batch` on the live sink in the parent, so mutations there *do* stick. Note which way that cuts. A sink developed and tested against `serial` will pass, then silently lose every write the first time it runs on a real pool. Write `prepare_batch` as if the copy is always thrown away, because under the executor you care about it is.

The converse is the one that costs. Anything your sink accumulates in `commit_batch` is re-pickled into *every chunk submitted after it*, so a sink that retains per-point state makes the run pay for its own output quadratically – exactly the cost this split exists to avoid. That state is also pickled on the dispatch thread while your writer thread may be midway through mutating it, so what the worker sees is a snapshot taken at no particular moment. Give your sink a `__getstate__` that drops whatever `prepare_batch` does not read:

```
    def __getstate__(self) -> dict[str, object]:
        state = dict(self.__dict__)
        state["_committed"] = {}   # accumulated at commit time; a worker
        return state               # neither needs it nor can keep it
```

`MemorySink` sheds its retained orders this way and `CsvSink` sheds both its compiled query and the order's point list.

If your sink commits in the parent (`supports_worker_commit()` returns `False`), consider also returning `True` from `supports_background_commit()`. Eleanor then runs your commits on a dedicated writer thread, so the dispatch loop can keep the worker pool fed while you write. The guarantee is narrow but firm: a *single* thread owns the sink for the run and is joined before `finalize_run`, so you never see concurrent commits and need no locking of your own – only the thread differs from the one that called `initialize` and `begin_run`. Return `False` if you hold anything thread-affine across those calls, such as a sqlite3 connection opened with `check_same_thread`, thread- local storage, or a C library with a per-thread context.

Also return `False` if your `commit_batch` is CPU-bound Python. A writer thread overlaps with the dispatch loop only while the commit *releases* the GIL; a commit that holds it contends with the dispatch loop and with the worker pool's result-unpickling thread instead, and measures slower than committing inline. In practice this means doing your row conversion in `prepare_batch` – where it is also parallel across workers – and leaving `commit_batch` as close to pure I/O as you can. Measure both ways before opting in.

None of this applies if `supports_worker_commit()` returns `True`: that sink commits in the worker, so there is no dispatch thread to get off and `supports_background_commit()` is never consulted. `PostgresSink` returns `False` here for that reason – not as a judgement about its commit cost.

Sinks share one run

A run may drive several sinks at once. Eleanor computes each point **once** and hands the same `Sequence[ComputeResult]` to every sink's `prepare_batch`, in the order the configuration lists them. Two obligations follow.

Do not mutate `results`, or anything reachable from them. A sink that writes to the compute graph is visible to every sink after it, and to any sink whose prepared payload aliases the graph rather than copying out of it – `PostgresSink`'s payload holds the point objects themselves. Eleanor cannot defend against this: duplicating a ~100k-object graph per sink is exactly the cost the prepare/commit split exists to avoid.

Own only what is yours. Your sink's id space, files, connections and counters are yours alone; nothing correlates them with another sink's. If your sink caches a resource keyed by something other than the sink instance – a connection pool keyed by its connection settings, say – two instances configured against the same target will share it, and the first to `finalize` must not tear it out from under the second.

Which raises the case that state cannot survive at all: two of your sinks pointed at *one* store. Nothing correlates their counters, their buffers or their file handles, so they corrupt each other. Names are what Eleanor deduplicates, and a name says nothing about where a sink points, so declare the target yourself and Eleanor will refuse the run:

```
    def target_key(self) -> object | None:
        return self.settings.filename.resolve()
```

Return anything that identifies the store; Eleanor compares keys with `==`, so they need not be hashable. The default is `None`, meaning "no exclusive target" – correct for a sink that discards or keeps results in memory, and the backwards-compatible answer for sinks written before this existed.

Sinks are addressed by the `name` in their config block, which defaults to `kind`. It keys the sink's resume token, labels its progress bar, and keys the ids `Eleanor.run` returns.

Declining to resume

`supports_resume()` defaults to `True`. Return `False` when there is nothing a resume token could name – a sink that draws to a window, or streams to a socket, retains no run to go back to. Eleanor then always passes `requested_id=None` and never requires a token for your sink, which matters because with several sinks active it demands one for every sink that *does* claim to support resuming.

```
from collections.abc import Sequence
from dataclasses import dataclass
from typing import cast

from eleanor.output.interface import (
    AbstractOutputSink,
    ComputeResult,
    WriteOutcome,
    require_int_order_id,
```

```
)
from eleanor.output.settings import OutputSinkSettings
from eleanor.plugin import ConfigurablePluginSpec


@dataclass(slots=True, frozen=True)
class MyPrepared:
    exit_code: int


class MySink(AbstractOutputSink[int]):
    def begin_run(self, order, *, requested_id=None):
        # The sink owns the id space, so ``requested_id`` arrives as the raw
        # token the caller passed to ``--order-id``. Parse it yourself, and
        # raise if it is malformed or names no run you hold -- resuming is an
        # explicit request, so starting a new run instead would lose it.
        if requested_id is None:
            return 0
        return require_int_order_id(requested_id, "my sink")

      def prepare_batch(self, order_id, results: Sequence[ComputeResult]) -
> Sequence[MyPrepared]:
        # Runs in a worker. One item per result, in order -- that pairing is
        # what lets commit_batch report outcomes[i] for results[i]. Record a
        # per-point failure here rather than raising, so one bad point does
        # not cost the whole chunk.
        return [MyPrepared(exit_code=r.point.exit_code) for r in results]

    def commit_batch(self, order_id, prepared: Sequence[object], progress=None)
-> list[WriteOutcome]:
        # Narrow back to your own payload type: Python cannot express "some
        # type the sink chose", so the cast is explicit and local.
        items = cast(Sequence[MyPrepared], prepared)
            return [WriteOutcome(exit_code=i.exit_code, committed=True) for i
in items]

    def finalize_run(self) -> None:
        return None

    def finalize(self) -> None:
        return None
```

```python
def _parse_settings(raw: dict) -> OutputSinkSettings:
    return OutputSinkSettings.from_dict(raw)


def _build_sink(settings: object) -> MySink:
    return MySink()


my_sink_spec = ConfigurablePluginSpec(
    parse_settings=_parse_settings,
    build=_build_sink,
    plugin_api_version=1,
)
```

Install the package, then reference it in Eleanor config:

```yaml
output:
  kind: my_sink
```

You can verify discovery with:

```
eleanor doctor
```

Inline plugins

Every extension point supports two non-packaged registration paths:

1. **Programmatic name registration.** Call `register_<kind>(name, spec)` from the host script, then reference the name in the order file or CLI:

   ```python
   from eleanor.navigator.registry import register_navigator
   from eleanor.plugin import SimplePluginSpec
   from my_script import MyNavigator

   register_navigator("my_nav", SimplePluginSpec(build=MyNavigator))
   ```

2. **Direct instance override.** `Eleanor.run` accepts already-constructed objects that bypass the registry entirely:

   ```python
   from eleanor import Eleanor

   Eleanor(config=config).run(
       order,
   ```

```
    simulation_size,
    navigator=my_navigator_instance,
    kernel=my_kernel_instance,
    output_sink=my_output_sink_instance,
)
```

The override, when supplied, short-circuits the registry lookup for that extension point only. The `executor` override must be supplied at construction time (`Eleanor(executor=my_executor)`), not per-run.

Writing a kernel plugin

Kernel plugins register a `ConfigurablePluginSpec` that bundles two entry points: parsing order-file settings and constructing the kernel at run time.

```
from eleanor.kernel.registry import register_kernel
from eleanor.plugin import ConfigurablePluginSpec
from my_project.kernel import MyKernel, MySettings

my_kernel_spec = ConfigurablePluginSpec(
    parse_settings=MySettings.from_dict,
    build=lambda settings: MyKernel(settings),
    plugin_api_version=1,
)
register_kernel("my_kernel", my_kernel_spec)
```

Commercial plugins

Entry points are pure metadata, so proprietary plugin wheels shipped through a private index (and optionally Cython-compiled) participate in discovery identically to open-source plugins. The registered factory can live inside a compiled `.so` and perform license verification on first use; Eleanor will surface any failure through the registry's standard warning/error path.

Collision policy

No shadowing is allowed:

- Registering under a built-in name (via `register_<kind>()` or an entry point) is always a hard error.
- Two entry points claiming the same name — whether built-in or not — is a hard error listing both offending entry-point values.
- Programmatic `register_<kind>()` calls that collide with an already-registered name are a hard error.

To replace a built-in at run time without touching the registry, use the direct-instance-override path on `Eleanor.run(kernel=..., ...)` or pass an executor at construction time via `Eleanor(executor=...)`.

The `ELEANOR_<KIND>_OVERRIDES` environment variable only affects API-version checks (downgrading version mismatches to warnings); it has no effect on name collision handling.

Writing a new extension point

If you add a new pluggable concept to Eleanor, instantiate another `PluginRegistry` with the appropriate spec shape and entry-point group. The generic primitive handles discovery, validation, collision resolution, and override semantics, so the new registry module stays small:

```
from eleanor.plugin import PluginRegistry

registry = PluginRegistry(
    kind='thing',
    entry_point_group='eleanor.things',
    override_env_var='ELEANOR_THING_OVERRIDES',
    builtin_names=frozenset({'default'}),
)

register_thing = registry.register
available_things = registry.available
get_factory = registry.get
```

`builtin_names` reserves names whose specs will arrive via entry-point discovery. In production registries `builtin_names` lists the names declared in `pyproject.toml`.

Then declare the built-in in `pyproject.toml` so discovery can load it:

```
[project.entry-points."eleanor.things"]
default = "eleanor.thing.factories:default_spec"
```